

LAB 5

Artworks Management System

Instructions

In this assignment, you will build a secure backend web application to manage a digital gallery of artworks (paintings), where:

- Multiple users can register and log in as artists or collectors.
- Authenticated users can create artworks.
- Each artwork is linked to an author (creator).
- Users can be granted **ownership permission** for artworks created by others.
- Authenticated users can view the list of artworks they **own**.
- EJS templates will be used to render artworks and categories.

This assignment includes REST API development and EJS-based frontend rendering.

Technologies Required

- **Backend:** Node.js, Express.js, MongoDB, Mongoose
- **Authentication:** JWT, bcryptjs
- **Frontend (views):** EJS template engine
- **Environment:** dotenv, cors
- **Tools:** Postman, MongoDB Compass, Visual Studio Code

Data Model Overview

Collections:

1. **users:** stores user login credentials
2. **categories:** stores artwork categories (e.g., Abstract, Landscape)
3. **artworks:** stores information about each painting
4. **ownerships:** stores ownership permission for users

Task Breakdown

Task 1: Project Initialization & Environment Setup

Description:

- Create a new Node.js project
- Install dependencies:

```
bash

npm install express mongoose dotenv cors bcryptjs jsonwebtoken ejs
```
- Create **.env** file:

```
env

PORT=6000
MONGO_URI=mongodb://localhost:27017/artworksdb
JWT_SECRET=your_secret_key
```

Test:

- Run `npm start`
- Visit: `http://localhost:6000`

Expected Output:

- Server runs successfully
- MongoDB connects without error

◆ Task 2: User Registration and Login (JWT)

📖 2.1 Register (POST /users/register)

- Accepts username and password
- Hash password with bcrypt
- Ensure username is unique

✓ Test:

json

```
POST /users/register
{
  "username": "collector01",
  "password": "secure123"
}
```

✓ Output:

json

```
{ "message": "User registered successfully" }
```

📖 2.2 Login (POST /users/login)

- Validate user credentials
- Return signed JWT token with userId and username

✓ Test:

json

```
POST /users/login
{
  "username": "collector01",
  "password": "secure123"
}
```

✓ Output:

json

```
{ "token": "eyJhbGciOi..." }
```

◆ Task 3: Artwork Management (Create, Update, Delete)

📖 3.1 Create Artwork (POST /artworks)

- Requires JWT token
- Body fields: title, description, imageUrl, categoryId
- Store createdBy as current user

✓ Test:

json

```
POST /artworks
Authorization: Bearer <token>
{
  "title": "Sunset Dream",
  "description": "A warm coastal sunset",
  "imageUrl": "https://example.com/sunset.jpg",
  "categoryId": "<category_id>"
}
```

✓ Output:

json

```
{ "message": "Artwork created successfully" }
```

3.2 Update Artwork (PUT /artworks/:id)

- Only the creator of the artwork can update it

 **Test:**

json

```
PUT /artworks/64aee300...
Authorization: Bearer <token>
{
  "title": "Updated Sunset",
  "description": "Edited description"
}
```

 **Output:**

json

```
{ "message": "Artwork updated successfully" }
```

3.3 Delete Artwork (DELETE /artworks/:id)

- Only the creator can delete their artwork

 **Test:**

http

```
DELETE /artworks/64aee300...
Authorization: Bearer <token>
```

 **Output:**

json

```
{ "message": "Artwork deleted successfully" }
```

Task 4: Grant Artwork Ownership

Description:

Allows a user (typically the creator or admin) to assign ownership of an artwork to another user. Ownership is stored in a separate collection.

Endpoint: POST /ownerships/grant

- Body: artworkId, userId

 **Test:**

json

```
POST /ownerships/grant
Authorization: Bearer <token>
{
  "artworkId": "<artwork_id>",
  "userId": "<target_user_id>"
}
```

 **Output:**

json

```
{ "message": "Ownership granted" }
```

Task 5: View Owned Artworks (Authenticated)

Description:

Users can view artworks they are authorized to own (granted via ownership collection). Requires authentication.

 **Endpoint: GET /users/me/owned-artworks**

 **Test:**

http

GET /users/me/owned-artworks
Authorization: Bearer <token>

 **Output:**

json

```
[
  {
    "title": "Sunset Dream",
    "categoryId": { "name": "Landscape" },
    "createdBy": { "username": "artist01" },
    "imageUrl": "https://example.com/sunset.jpg"
  }
]
```

Task 6: Render Views with EJS

6.1 View All Categories (GET /categories/view)

- Render categories in HTML list

 **Output (HTML):**

html

```
<ul>
  <li>Landscape</li>
  <li>Abstract</li>
</ul>
```

6.2 View All Artworks (GET /artworks/view)

- Render table with:
 - Title
 - Category name (populated)
 - Author name

 **Output (HTML):**

html

```
<table>
  <tr><th>Title</th><th>Category</th><th>Author</th></tr>
  <tr>
    <td>Sunset Dream</td>
    <td>Landscape</td>
    <td>artist01</td>
  </tr>
</table>
```

6.3 View Owned Artworks (GET /users/me/owned-artworks/view)

- Requires login
- Render HTML table of artworks the user owns

Test Cases Summary

#	Scenario	Endpoint	Auth	Expected
1	Register user	POST /users/register		Success message
2	Login	POST /users/login		JWT token

3	Create artwork	POST /artworks	✓	Created
4	Update own artwork	PUT /artworks/:id	✓	Updated
5	Delete artwork	DELETE /artworks/:id	✓	Deleted
6	Grant ownership	POST /ownerships/grant	✓	Granted
7	View owned artworks	GET /users/me/owned-artworks	✓	JSON list
8	View owned artworks (guest)	GET /users/me/owned-artworks	✗	401 Unauthorized
9	View categories	GET /categories/view	✗	HTML
10	View all artworks	GET /artworks/view	✗	HTML <table>
11	View owned artworks (HTML)	GET /users/me/owned-artworks/view	✓	HTML <table>

Sample JSON Data (for MongoDB import)

categories.json

json

```
[
  { "_id": { "$oid": "64aee200c438b927a9cfa222" }, "name": "Landscape" },
  { "_id": { "$oid": "64aee201c438b927a9cfa223" }, "name": "Abstract" }
]
```

users.json

json

```
[
  {
    "_id": { "$oid": "64aee111c438b927a9cfa111" },
    "username": "artist01",
    "password": "$2a$10$....",
    "createdAt": { "$date": "2025-07-01T10:00:00.000Z" }
  }
]
```

artworks.json

json

```
[
  {
    "_id": { "$oid": "64aee300c438b927a9cfa444" },
    "title": "Sunset Dream",
    "description": "A beautiful sunset over the ocean",
    "imageUrl": "https://example.com/sunset.jpg",
    "categoryId": { "$oid": "64aee200c438b927a9cfa222" },
    "createdBy": { "$oid": "64aee111c438b927a9cfa111" },
    "createdAt": { "$date": "2025-07-01T11:00:00.000Z" }
  }
]
```

ownerships.json

json

```
[
  {
    "_id": { "$oid": "64ccccccc438b927a9cfa999" },
    "artworkId": { "$oid": "64aee300c438b927a9cfa444" },
    "userId": { "$oid": "64aee222c438b927a9cfa123" },
    "grantedAt": { "$date": "2025-07-10T10:00:00.000Z" }
  }
]
```

 Submission Requirements

- Submit a .zip file named:

css

[StudentID]_[FullName]_assignment_artworks.zip

- Inside:
 - source code with comments
 - screenshots: VSCode structure, Postman tests, HTML output
 - report: .docx or .pdf
 - sample_data/: .json files for import

-